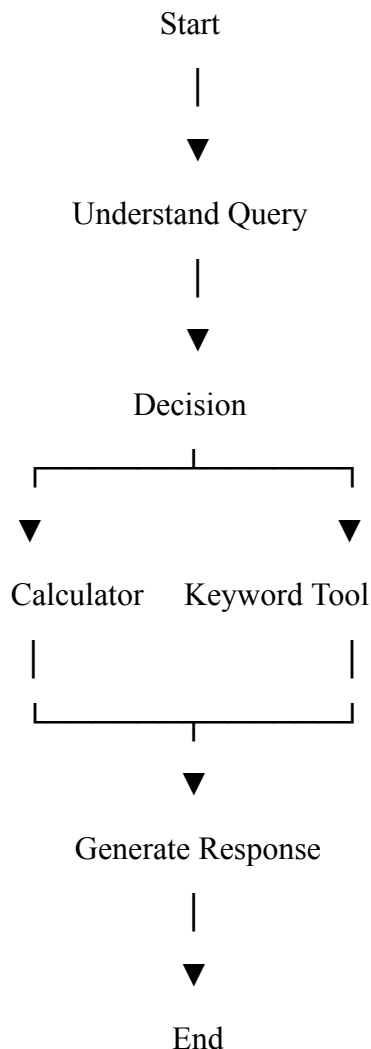


Single Agent Systems & Agent Pipelines

- Quiz

1. Explain the concept of a stateful directed graph in agent pipelines. How does it differ from a simple linear pipeline?

A stateful directed graph is a workflow where the agent can remember information from previous steps while completing a task. It is made up of nodes connected by edges, and the workflow can branch into different paths or even repeat certain steps if needed. A linear pipeline, on the other hand, always follows the same fixed sequence from start to finish without storing previous information or changing its path.



2. Describe the role of nodes and edges in an agent workflow. Give an example of each.

Nodes represent the individual tasks performed by the agent, such as analyzing a query, using a calculator, or generating a response. Edges connect these nodes and define how the workflow moves from one step to another. For example, a "Calculator Tool" is a node, while the connection from the query analysis step to the calculator is an edge.

3. What is conditional routing in an agent system? Design a simple rule-based routing logic for three different query types.

Conditional routing means sending a user query to the most suitable tool based on its type or intent. For example:

- If the query contains "**calculate**", send it to the **Calculator Tool**.
- If the query contains "**keywords**", send it to the **Keyword Extraction Tool**.
- For all other queries, send them to the **General Response Module**.

This helps the agent choose the correct action for different types of requests.

4. Why are cycles (loops) important in agent pipelines? Provide a use case where a retry loop is necessary.

Loops allow the agent to repeat a task until it gets the expected result. They are useful when an operation fails or needs another attempt. For example, if an API request fails because of a temporary network issue, the agent can retry instead of stopping immediately. This makes the system more reliable.

5. Explain how a single-agent system can simulate multi-agent behavior internally.

A single-agent system can perform different roles one after another. It may first understand the user's query, then decide which tool to use, and finally generate the answer. Even though only one agent is running, it behaves like multiple agents working together by handling different responsibilities in separate steps.

6. What are JSON schema tools? How do they help in structuring tool inputs and outputs?

JSON schema tools define a standard format for exchanging data between the agent and different tools. They specify the required fields, data types, and expected structure of the input and output. This helps avoid errors, keeps the data organized, and makes communication between components more reliable.

7. Compare sequential tool calls and parallel tool calls. When would you prefer one over the other?

Sequential tool calls are performed one after another because each step depends on the previous one. Parallel tool calls run multiple independent tasks at the same time. Sequential execution is useful when there is a dependency between tasks, while parallel execution is preferred when tasks are independent, as it saves time and improves efficiency.

- **SEQUENTIAL:**

User Query

|



Tool A

|



Tool B

|

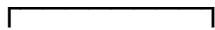


Tool C

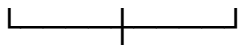
- **PARALLEL:**

User Query

|



Tool A Tool B Tool C



Final Response

8. How would you implement error handling in a tool-using agent? Provide at least two strategies.

Error handling helps the agent continue working even when something goes wrong. One way is to use **try-except** blocks to catch errors and display meaningful messages instead of crashing. Another approach is to implement a **retry mechanism**, where the agent automatically tries the failed task again after a short delay. Keeping logs of errors is also helpful for debugging later.

9. What is trajectory evaluation in agent systems? Why is it important beyond just checking final output?

Trajectory evaluation means checking every step the agent takes while solving a problem instead of only looking at the final answer. It helps us understand whether the agent selected the right tools, followed the correct reasoning process, and made good decisions throughout the workflow. This makes debugging and improving the agent much easier.

10. Define task completion rate and cost metrics. How would you measure and optimize them in a real-world system?

Task completion rate is the percentage of tasks that the agent completes successfully. Cost metrics measure the resources used, such as processing time, API calls, or overall computational cost. These can be tracked over multiple tasks. To improve them, we can make tool selection more efficient, avoid unnecessary tool calls, and optimize the workflow so that tasks are completed successfully with fewer resources.

- **Agent Pipeline**

